

3 | Codierung

Wenn wir ein Foto anschauen, sehen wir ein Bild. Wenn wir ein Lied hören, hören wir Musik. Ein Computer nimmt diese Dinge jedoch nicht so wahr wie wir. Er hat ja keine Augen oder Ohren. Für ihn besteht ein Foto, ein Lied oder ein Video aus einer langen Folge von 0 und 1, weil das ist ja seine Sprache. Jede einzelne 0 oder 1 ist dabei ein **Bit**.

Damit wir aber nicht solch für unsere Augen nichts aussagenden Ansammlungen von 0 und 1 anschauen müssen, übersetzt der Computer diese binären Anordnungen in Pixel oder Buchstaben. Diese Übersetzung zwischen menschenverständlichem und computerverständlichem bezeichnet man als **Codierung** und **Decodierung**.



1. Textcodierung

Bei der Textcodierung geht es also darum, wie der Computer Text (Buchstaben, Emojis, ...) speichern, darstellen und/oder übertragen möchte. Bevor wir unsere heutigen Computer hatten, gab es schon erste Technologien, die fähig waren, Buchstaben und Zahlen über Ferne zu übertragen. Schauen wir uns mal folgendes Beispiel an:

Das Fernschreibgerät mit Telex



Das Gerät das Sie links sehen ist ein Fernschreibgerät, das mit dem Telex System mittels elektrischer Signale weltweit Nachrichten austauschen konnte. Quasi ein Vorgänger der schriftlichen Kommunikation, wie wir sie heute auf unseren Smartphones kennen. Unten an der Tastatur konnte der Benutzer eine Nachricht schreiben, welche dann an einen anderen Benutzer gesendet werden konnte. Der konnte dann die Nachricht ausgedruckt auf seinem Stück Papier sehen. Aber was passierte zwischen Schreiben und Drucken im Hintergrund?

Um diese Abfolge zwischen geschriebenem Buchstaben, Übertragung und ausgedrucktem Buchstaben zu verstehen, betrachten Sie die Abbildung 1.

Die Tabelle zeigt, wie die einzelnen Zeichen codiert wurden. Schwarz bedeutet, dass Strom fließt, weiss bedeutet, dass kein Strom fließt. Wenn Sie beispielsweise den Buchstaben A eingeben, sendet der Fernschreiber zunächst ein Startimpuls. Dieses signalisiert dem empfangenden Gerät, dass nun ein neues Zeichen übertragen wird. Anschliessend werden fünf Bits übertragen. Beim A lautet die Folge:

Strom – Strom – kein Strom – kein Strom – kein Strom

Zum Abschluss folgt ein Stoppimpuls, das das Ende des Zeichens signalisiert.

Der empfangende Fernschreiber liest diese Folge und vergleicht sie mit derselben Codetabelle. Dadurch erkennt er, dass die übertragene Zeichenfolge für den Buchstaben A steht, und kann diesen ausdrucken. Danach beginnt die Übertragung des nächsten Zeichens.

Dieses Verfahren wird als Start-Stopp-Verfahren bezeichnet. Die Buchstaben wurden also nicht direkt übertragen, sondern zunächst in eine Folge von zwei möglichen Zuständen (Strom oder kein Strom) **codiert**.

Aufgabe 1

Für einen Buchstaben gibt es 1 Startimpuls, 5 Impulse (5 Bit) für das Zeichen und 1.5 Stoppimpulse (eigentlich kein Bit, sondern eine Zeit). Das Ganze dauert 150ms. Wie lange dauert ein einzelner Impuls? Wie viele Zeichen pro Minute sind möglich?

NR. VAN DE COMBI- NATIE	LETTERS	CIJFERS	NR. VAN DE ELEMENTEN						
			START	1	2	3	4	5	STOP
1	A	—							
2	B	?							
3	C	:							
4	D								
5	E	3							
6	F								
7	G								
8	H								
9	I	8							
10	J	DEL							
11	K	(							
12	L	)							
13	M	.							
14	N	*							
15	O	9							
16	P	0							
17	Q	!							
18	R	4							
19	S	'							
20	T	5							
21	U	7							
22	V	=							
23	W	2							
24	X	/							
25	Y	6							
26	Z	+							
27	TERUGLOOP WAGEN								
28	NIEUWE REGEL								
29	LETTERS								
30	CIJFERS								
31	TUSSENRUIMTE								
32	5 x STROOMLOOS								

Abbildung 1: Der Telex Code

Der Morsecode

Ein anderes System, um Texte über weite Strecken zu senden ist der Morsecode. Jeder Buchstabe wird durch eine Folge aus Punkten und Strichen dargestellt. Ein Punkt steht für ein kurzes Signal, ein Strich für ein langes Signal (siehe die Beispielbuchstaben in Abbildung 2). Die Zeichen sind unterschiedlich lang. Ein E besteht beispielsweise aus einem einzelnen Punkt und das J aus einem Punkt und drei Strichen.

A — • —	G — • •	U • • —
B — • • •	H • • • •	V • • • —
C — • — •	I • •	W • — —
D — • •	J • — —	X • — • —
E •	K • — •	Y — • — •
F • • • —	L • — • •	Z — — • •

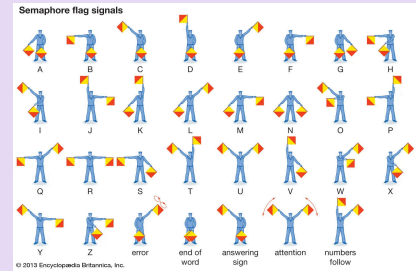
Abbildung 2: Morsecode

Andere historische Textcodierungen

Das Telex System oder der Morsecode waren nicht die einzigen, die es Menschen ermöglichte, Texte über weite Strecken zu senden. Der **Semaphor Code** wurde zum Beispiel in der Schifffahrt verwendet, um zwischen Booten über weitere Distanzen zu kommunizieren. Die Buchstaben werden durch unterschiedliche Positionen von zwei Flaggen dargestellt. Jede Stellung der Arme entspricht einem bestimmten Buchstaben (siehe rechts).



Ein ähnliches Prinzip wurde auch bei den optischen Telegrafen verwendet. In Frankreich entstand Ende des 18. Jahrhunderts ein Netz aus Telegrafentürmen, die über grosse Distanzen miteinander verbunden waren. Auf jedem Turm wurden bewegliche Signalarme in verschiedene Positionen gebracht. Die nächste Station beobachtete das Signal und gab es an die nächste Station weiter. Auf diese Weise konnten Nachrichten über grosse Entfernungen übertragen werden.



Keine grosse Überraschung, aber sowohl der Telex sowie auch der Morsecode sind heute von modernen Technologien überholt worden. Die Frage ist aber, warum war man mit diesen Lösungen nicht zufrieden?

Aufgabe 2

Telex und Morsecode ermöglichten die Übertragung von Nachrichten über grosse Distanzen. Doch beide Verfahren haben ihre Schwachstellen. Versuchen wir sie herauszufinden.

Beim Telex wird jedes Zeichen mit 5 Bits codiert und beim Morsecode sind die Zeichen unterschiedlich lang.

- Wie viele verschiedene Zeichen können mit 5 Bits wie beim Telex grundsätzlich dargestellt werden?
- Reicht diese Anzahl aus, um alle Zeichen darzustellen, die wir heute beim Schreiben benötigen?
- Sie möchten die Nachricht «GE» übertragen. Schreiben Sie den entsprechenden Morsecode auf.
- Könnte ein Empfänger anhand dieser Folge eindeutig die einzelnen Zeichen erkennen? Was müsste noch zusätzlich gesendet werden, damit die Eindeutigkeit gewährleistet ist?
- Welche Eigenschaften müsste eine neue Zeichencodierung besitzen, damit sie die Schwachstellen von Telex und Morsecode möglichst gut löst?

1.1. ASCII Codierung

1963 wurde schliesslich der **ASCII-Code** eingeführt. Dieser verzichtete auf Start- und Stoppbits und wurde ursprünglich für alle Zeichen einheitlich auf 7 Bit festgelegt.

Diese Zeichen beinhalten:

- 33 nicht druckbare Steuerzeichen
- 95 echte Zeichen (siehe rechts)

	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	{		}	~	

Diese Codes für jedes einzelne Zeichen müssen Sie nicht auswendig lernen. Für das gibt es ASCII-Tabellen (Abbildung 3), welche Sie auch an der Prüfung benutzen dürfen. Schauen wir uns eine solche Tabelle konkret an ein paar Beispielen an:

ASCII-Tabelle lesen

In der Tabelle sehen Sie, dass dem Buchstaben A der Wert 65 zugeordnet ist. Diese Zuordnung wurde festgelegt und jedes Zeichen erhält dadurch einen eindeutigen Code. Das kleine a hat zum Beispiel den Wert 97. Das sind Dezimalzahlen. Der Computer speichert die Informationen jedoch nicht als Dezimalzahlen, sondern binär.

Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
38	26	046	&	&	70	46	106	F	F	102	66	146	f	f

In der Tabelle finden Sie deshalb auch den hexadezimalen Wert. Die Hexadezimalschreibweise ist kompakter und benötigt weniger Platz als die vollständige binäre Darstellung. Beim Buchstaben A ist der hexadezimale Wert beispielsweise 41₁₆. In Binärschreibweise entspricht dies: 100'0001₂ (7 Bit!).

5	5	005	ENQ	{enquiry}
6	6	006	ACK	{acknowledge}
7	7	007	BEL	{bell}
8	8	010	BS	{backspace}
9	9	011	TAB	{horizontal tab}
10	A	012	LF	{NL line feed, new line}

Die sichtbaren Zeichen beginnen in der Tabelle erst ab dem Wert 32. Davor befinden sich sogenannte Steuerzeichen. Diese lösen bestimmte Aktionen aus. Einige davon sind Ihnen wahrscheinlich bereits bekannt: Mit backspace löschen Sie beispielsweise ein Zeichen, und mit new line erzeugen Sie einen Zeilenumbruch.

Für die folgenden Aufgaben brauchen Sie die ASCII-Tabelle, die Sie direkt unterhalb einsehen können.

Aufgabe 3

Schreiben Sie den folgenden Satz in ASCII, indem Sie die Codes in **dezimaler** Schreibweise hinschreiben:

Macht Informatik Spass?

Aufgabe 4

Schreiben Sie den folgenden Satz in ASCII, indem Sie die Codes in **binärer** Schreibweise hinschreiben:

Heute ist es schön 😊

ASCII-Tabelle

Dec	Hex	Character
0	00	NUL
1	01	SOH
2	02	STX
3	03	ETX
4	04	EOT
5	05	ENQ
6	06	ACK
7	07	BEL
8	08	BS
9	09	HT
10	0A	LF
11	0B	VT
12	0C	FF
13	0D	CR
14	0E	SO
15	0F	SI
16	10	DLE
17	11	DC1
18	12	DC2
19	13	DC3
20	14	DC4
21	15	NAK
22	16	SYN
23	17	ETB
24	18	CAN
25	19	EM
26	1A	SUB
27	1B	ESC
28	1C	FS
29	1D	GS
30	1E	RS
31	1F	US
32	20	(Space)
33	21	!
34	22	"
35	23	#
36	24	\$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.

Dec	Hex	Character
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=
62	3E	>
63	3F	?
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z
91	5B	[
92	5C	\
93	5D	]

Dec	Hex	Character
94	5E	^
95	5F	_
96	60	`
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g
104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z
123	7B	{
124	7C	
125	7D	}
126	7E	~
127	7F	DEL

Abbildung 3: Die ASCII-Tabelle mit allen Zeichen von 0 bis 127 (insgesamt 128 Zeichen).

ASCII scheint also ebenfalls nicht ganz vollständig unsere modernen Anforderungen zu erfüllen. Zwei der Zeichen aus den Sätzen von den Aufgaben sind nicht in der Tabelle enthalten... Haben Sie eine Ahnung, warum Zeichen mit Akzent (é, â, ö, ñ) nicht enthalten sind? Die Antwort liegt im A von ASCII.

1.2. Weitere Codierungen

Eine naheliegende Idee war, ASCII zu erweitern: Statt 7 Bits verwendet man 8 Bits. Genau das machte IBM beispielsweise mit **Code Page 437**. Die ersten 128 Zeichen blieben unverändert und entsprachen ASCII. Die zusätzlichen 128 Zeichen wurden mit weiteren Buchstaben, Symbolen und grafischen Zeichen gefüllt.

Aufgabe 5

Wie viele verschiedene Zeichen können mit 8 Bits dargestellt werden? Wie viele davon kommen im Vergleich zu 7 Bits hinzu?

Wenn wir uns die Code Page 437 in Abbildung 4 genauer betrachten, können wir schnell überprüfen, dass die Zeichen für die ASCII Werte dieselben sind wie vorher:

Nehmen wir mal den Grossbuchstaben A. In der Tabelle sehen wir, dass er in Zeile 4, Spalte 1 ist. Das kann man so lesen, dass der Hexadezimalwert von A 41_{16} ist. Das ist derselbe Wert, den A in der Tabelle in Abbildung 3 hat.

Nun konnte man den Computer auch in Ländern mit Sprachen mit Umlauten (z.B. Schweiz) verkaufen. Ein paar der Zeichen sehen aber sehr speziell aus. Diese Linien, die man z.B. in der Zeile C sieht, dienten damals zur Hilfe, um grafisch zu malen. So sehen Sie in Abbildung 5 die rot eingezeichneten Quadrate entsprechen den Zeichen B3, C1 und C7 in der Code Page 437.

Aufgabe 6

Ein Problem bleibt aber. 128 zusätzliche Zeichen reichen immer noch nicht für alle Sprachen auf der Welt. Kennen Sie ein Alphabet wessen Zeichen nicht in dieser Code Page 437 enthalten sind?

Deshalb wurden weitere Codepages entwickelt. Codepage 850 war beispielsweise stärker auf westeuropäische Sprachen ausgerichtet und enthielt dafür andere Zeichen als Codepage 437 (wobei die ursprünglichen ASCII-Zeichen dieselben blieben).

Code Page 437

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F
0-	*	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣
1-	1	2	3	4	5	6	7	8	9	:	<	=	>	?		
2-	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	
3-	0	1	2	3	4	5	6	7	8	9	:	<	=	>	?	
4-	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	
5-	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	
6-	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	
7-	p	q	r	s	t	u	v	w	x	y	z	{		}	~	
8-	ç	ü	é	â	ä	å	å	å	å	å	å	å	å	å	å	
9-	é	æ	æ	ô	ö	ö	ö	ö	ö	ö	ö	ö	ö	ö	ö	
A-	á	í	ó	ú	ñ	ñ	ñ	ñ	ñ	ñ	ñ	ñ	ñ	ñ	ñ	
B-	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	
C-	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	
D-	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	␣	
E-	α	β	Γ	π	Σ	σ	ρ	τ	Φ	Θ	Ω	δ	ε	φ	ε	η
F-	±	≥	≤	ƒ	J	z	~	°	°	°	°	°	°	°	°	
	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F

Used first in IBM Monochrome Display Adapter in 1981.

Abbildung 4: Code Page 437, wie sie IBM 1981 darstellte. Die Reihen und Spalten sind Hexadezimalzahlen.

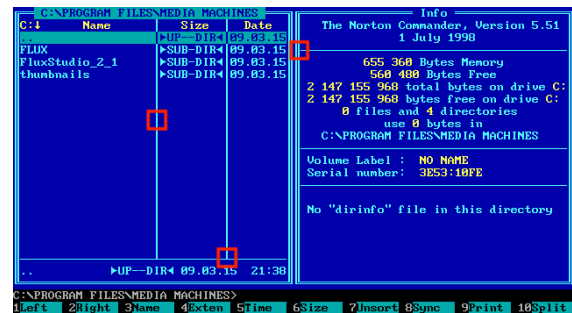


Abbildung 5: Der Norton Commander war eine frühere Version ihres heutigen Explorers/Finders auf ihrem Windows/Mac.

437	Die ursprüngliche Zeichensatztabelle des IBM-PC	852	Slawische Sprachen (Latin-2), zentraleuropäische und osteuropäische Sprachen	862	Hebräisches Alphabet
720	Arabisches Alphabet	855	Kyrillisches Alphabet	863	Lateinisches Alphabet mit französischen Sonderzeichen
737	Griechisches Alphabet	857	Türkisches Alphabet	864	Arabisches Alphabet
775	Estrnisches Alphabet, Litauisches Alphabet und Lettisches Alphabet	858	„Multilingual“ mit Eurozeichen	865	Dänisch und Norwegisch – unterscheidet sich von 437 nur durch Ø (ø) anstelle von ¥ und ¢
819	„Latin-1“, entspricht ISO 8859-1	860	Lateinisches Alphabet mit portugiesischen Sonderzeichen	866	Kyrillisches Alphabet
850	„Multilingual (DOS-Latin-1)“, westeuropäische Sprachen	861	Isländisches Alphabet	869	Griechisches Alphabet

Abbildung 6: Beispiele verschiedener Codepages. Sie finden noch einige mehr unter <https://de.wikipedia.org/wiki/Zeichensatztabelle>

Jetzt stellen Sie sich nun aber mal vor, Sie schreiben einen Text auf Ihrem Computer in der Schweiz (Codepage 850). Anschliessend schicken Sie diesen Text an jemanden, dessen Computer griechisch eingestellt ist (Code Page 869). Die hexadezimalen Zahlen werden zwar korrekt übertragen – aber der Empfänger interpretiert sie mit einer anderen Tabelle. Aus einem Zeichen kann dadurch plötzlich ein völlig anderes Zeichen werden.

Diese falsche Interpretation können Sie in Abbildung 7 sehen: Die Linien, welche in Abbildung 5 noch schön dargestellt wurden, entsprechen jetzt zum Teil dem Zeichen ä.

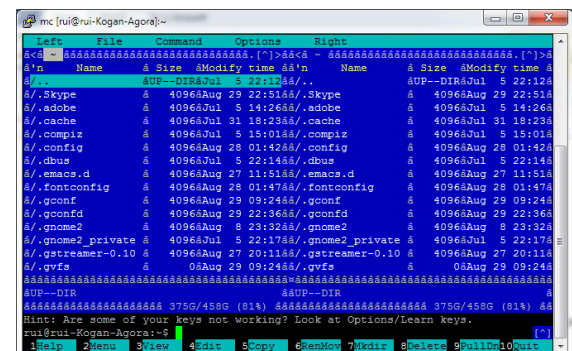


Abbildung 7: Der Norton Commander mit einer falschen Darstellung der Linien.

1.3. Unicode und UTF

Die vielen verschiedenen Codepages führten zu einem grossen Problem: Es gab zwar immer mehr Zeichen, aber nicht überall dieselben. Ein Zeichen konnte auf einem Computer etwas anderes bedeuten als auf einem anderen. Es brauchte deshalb eine einheitliche Zeichencodierung, die möglichst alle Zeichen der Welt umfasst.

Die Idee dahinter ist zunächst einfach: Wenn wir mehr Bits verwenden, können wir viel mehr verschiedene Zeichen darstellen. Das sieht man ganz schnell, wenn man sich die Auswirkungen der Zweierpotenzen anschaut:

$2^7 = 128$	mögliche Zeichen bei 7 Bit (ASCII)
$2^8 = 256$	mögliche Zeichen bei 1 Byte (Codepage)
$2^{16} = 65'536$	mögliche Zeichen bei 2 Bytes
$2^{32} = 4'294'967'296$	mögliche Zeichen bei 4 Bytes

Genau hier setzt **Unicode** an. Unicode ist ein weltweit einheitliches System, bei dem Zeichen aus verschiedensten Sprachen eine eindeutige Nummer erhalten. Dazu gehören beispielsweise lateinische, griechische und chinesische Schriftzeichen, aber auch mathematische Symbole, Sonderzeichen und Emojis (siehe Abbildung 8).

Unicode beantwortet also einfach mal, welches Zeichen gehört zu welcher Nummer... Aber wie wird das nun codiert?



Abbildung 8: Ein paar Beispiele aus dem Unicode Verzeichnis.

Hier kommt **UTF-8** ins Spiel. UTF steht für Unicode Transformation Format. UTF-8 ist eine Methode, Unicode-Zeichen so in Bytes umzuwandeln, dass häufig verwendete Zeichen möglichst wenig Speicherplatz benötigen. Dabei werden je nach Zeichen 1 bis 4 Bytes verwendet. Das erinnert zunächst an den Morsecode: Wenn Zeichen unterschiedlich lang sind, stellt sich die Frage, wie der Computer weiss, wo ein Zeichen beginnt und endet?

UTF-8 löst dieses Problem durch ein festgelegtes Muster für die einzelnen Bytes:

- Ein Zeichen mit 1 Byte beginnt mit 0.
- Ein Zeichen mit 2 Byte beginnt mit 110.
- Ein Zeichen mit 3 Byte beginnt mit 1110.
- Ein Zeichen mit 4 Byte beginnt mit 11110.
- Alle nicht-führenden Zeichen beginnen mit 10.

Eselsbrücke Zug

Man kann sich das wie einen Zug vorstellen: Das erste Byte ist die Lokomotive und zeigt an, wie viele Wagen zum Zug gehören (3 Einsen bedeuten insgesamt 3 Wagen). Die folgenden Bytes sind die Wagen und beginnen immer mit 10. So weiss der Computer, wann ein Zeichen endet.

Überlegung 💡

Eine Möglichkeit wäre, jedes Zeichen immer mit 4 Byte zu speichern. Das wäre aber sehr verschwenderisch! Ein Text mit 100 Zeichen würde dann bereits 400 Byte benötigen – auch wenn viele dieser Zeichen mit deutlich weniger Platz auskommen würden (welche z.B.?)

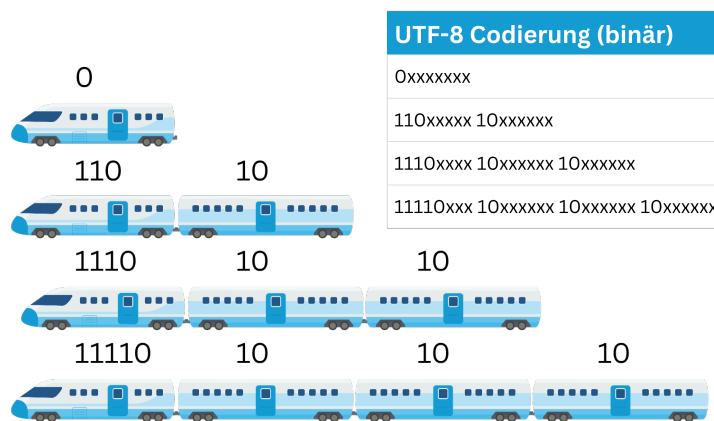


Abbildung 9: UTF-8 kann man sich wie einen Zug vorstellen, bei welchem der führende Wagen einen bestimmten Code hat und die folgenden immer 10.

UTF-8 wurde so entwickelt, dass die ersten 128 Unicode-Zeichen den ASCII-Zeichen entsprechen. Diese benötigen deshalb weiterhin nur ein Byte. Erst bei weiteren Zeichen werden zwei, drei oder vier Bytes verwendet.

Damit verbindet UTF-8 die grosse Zeichenvielfalt von Unicode mit einer platzsparenden Speicherung und ist heute eine sehr verbreitete Möglichkeit, Unicode-Zeichen zu speichern und zu übertragen.

Immer Aktuell

Unicode wird laufend erweitert. Wenn Sie beispielsweise davon hören, dass ein neues Emoji eingeführt wird, muss dieses Emoji einen eigenen, neuen Platz im Unicode-System erhalten.

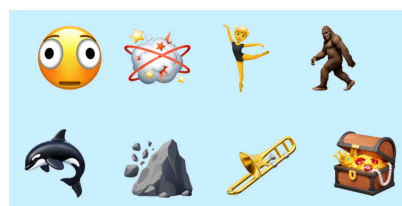


Abbildung 10: Im Frühling 2026 neu eingeführte Emojis

UTF-8 Codierungsbeispiele



Das Zeichen ö ist im Unicode Verzeichnis an der Stelle 00F6₁₆. Das ist eine 246₁₀ oder auch eine 1111'0110₂ (8 Bit). Das bedeutet, man braucht 8 Bit für die UTF-8-Codierung. Das entspricht einem Zug mit zwei Wagen (ein Zug mit nur einem Wagen hat 7 Bit zur Verfügung - das passt also nicht). Das heisst, die Codierung von ö hat sicher schon mal die Form 110x'xxxx 10xx'xxxx. Jetzt füllt man einfach die obige binäre Zahl von rechts her in diese Form ein und füllt die übrigen x mit 0 auf:

$$1111'0110_2 \rightarrow 1100'0011 \ 1011'0110_2$$

Das Emoji mit der Sonnenbrille ist an der 128'526. Stelle (HEX: 1F60E). Das in binär übersetzt ergibt die Zahl 0001'1111'0110'0000'1110₂ (20 Bit). Für das braucht man einen Zug mit insgesamt 4 Wagen, damit man die x besetzen kann. Der hat die Form 1111'0xxx 10xx'xxxx 10xx'xxxx 10xx'xxxx. Analog wie oben befüllt man von rechts her die Form und füllt die übrigen Stellen mit 0:



$$1111'0000 \ 1001'1111 \ 1001'1000 \ 1000'1110_2$$

Hier folgen genügend Aufgaben, um das Codieren mit den gelernten Codierungen (ASCII und UTF-8) zu üben.

Aufgabe 7

Codieren Sie die ASCII Nachricht hexadezimal:

Hinter 100 Hecken hocken 100 Hasen!

Aufgabe 8

Erklären Sie in Ihren eigenen Worten, was mit «nicht druckbaren Steuerzeichen» gemeint ist in der ASCII Tabelle.

Aufgabe 9

Decodieren Sie die beiden mit ASCII codierten Texte:

- 110'0100 110'1111 110'0111
- 011'1010 111'1011 010'1001
- 011'0010 011'1000 010'0000 010'1011 010'0000
011'0011 011'0111 010'0000 011'1101 010'0000
011'0110 011'0101

Aufgabe 10

Finden Sie die einzelnen Unicode Zeichen:

0100'1000 0110'0101 0111'1001 1111'0000
1001'1111 1001'1000 1000'1001 1100'0011
1010'0100 0110'1000 0110'1101 1111'0000
1001'1111 1000'1111 1000'0001

Aufgabe 11

Gegeben sind jeweils Zeichen und ihre jeweilige Stelle im Unicode-Verzeichnis. Codieren Sie sie mit UTF-8. Lassen Sie zur besseren Lesbarkeit zwischen den Bytes genügend Abstand.

- Q: 81
- 🐛: 128169
- ♥: 9829
- ü: 252

Aufgabe 12

Welches ist der binäre Unicode, der in den folgenden UTF-8-Codes codiert sind? Schreiben Sie den Unicode mit so vielen binären Stellen, wie nötig sind und lassen Sie zwischen den Bytes genügend Abstand.

- 1110'0010 1000'0010 1010'1100
- 1111'0000 1001'1111 1001'0000 1011'0110

Aufgabe 13

Lesen Sie die relevanten Bits für jedes UTF-8 codierte Symbol. Rechnen Sie sie dann ins Hexadezimalsystem um und recherchieren Sie im Internet, um welches Unicode Zeichen es sich handelt.

- 1111'0000 1001'1111 1000'1111 1000'0001
- 1100'0011 1010'0100

Aufgabe 14

Was wird hier codiert?

- 0100'0111 0110'0101 0110'1101 1100'0011
1011'1100 0111'0011 0110'0101
- 0111'0110 0110'1111 0110'1001 0110'1100
1100'0011 1010'0000

2. Bildercodierung

Für uns heutzutage ist es normal, dass unser Computer (oder sonstigen elektronischen Geräte) ohne Mühe Bilder darstellen kann. Wie wir im Kapitel vorher gesehen haben, war das früher überhaupt nicht selbstverständlich. Da hat man noch mit Textzeichen getrickst, um einfache Linien darstellen zu können.

Ein bekanntes Beispiel früherer solcher Darstellungen von Bildern ist sogar die eigene Kunstrichtung «ASCII-Art», bei der man Bilder nur mit ASCII-Zeichen dargestellt hat (siehe Abbildung 11).

Die heutigen Geräte haben also eine andere Möglichkeit, Bilder zu codieren (in 0 und 1 darzustellen), damit wir sie mit unserem Auge als Bilder wahrnehmen können. Dabei sind kaum Grenzen gesetzt und wir können an den Bildschirmen von Fotos bis zu digitalen Zeichnungen alles sehen.

Doch wie speichert jetzt nun ein Computer solche Bilder? Es gibt dafür verschiedene Formate. Wir werden uns 2 Konzepte anschauen: Die Rastergrafik und die Vektorgrafik. Die erstere ist den Meisten intuitiv vertraut.

Rastergrafik

Eine **Rastergrafik**, auch **Pixelgrafik** genannt, ist eine Form der Beschreibung eines Bildes in Form von computerlesbaren Daten.

Solche Grafiken bestehen aus einer rasterförmigen Anordnung von sogenannten Pixeln (Bildpunkten, siehe Abbildung 14), denen jeweils eine Farbe zugeordnet ist. Die Hauptmerkmale einer Rastergrafik sind:

- **Bildgrösse:** Breite + Höhe gemessen in Pixeln (auch *Bildauflösung* genannt)
- **Farbtiefe:** wie viele mögliche Werte ein Pixel annehmen kann

Schwarz-Weiss-Bilder

Schwarz-Weiss Bilder lassen sich sehr einfach mithilfe von Bits direkt darstellen. Schwarz zum Beispiel mit 1 und weiss mit 0 (oder umgekehrt). Soche einfachen Darstellungen kann der Computer im PBM (Portable BitMap/.pbm) Format speichern. Heutzutage ist das aber nicht mehr ein so weit verbreitetes Format und eignet sich besonders für Spielereien mit kariertem Papier.

Im Beispiel in Abbildung 15 ist die Bildgrösse 5x5 Pixel und pro Pixel wird 1 Bit gespeichert (0 oder 1): Man spricht also auch von einer 1-Bit-Farbtiefe.

Graustufenbilder

Was ist nun, wenn wir verschiedene Helligkeitsstufen zwischen Schwarz und Weiss darstellen möchten? Also alle Graustufen dazwischen. Dafür benötigen wir mehr Bits pro Pixel. Ein typisches Graustufenbild verwendet 8 Bit pro Pixel.

Mit 8 Bit können insgesamt $2^8 = 256$ verschiedene Werte dargestellt werden. Ein Pixel kann also 256 verschiedene Helligkeitsstufen annehmen: von Schwarz (0000'0000) über verschiedene Graustufen bis hin zu Weiss (1111'1111), siehe Abbildung 16. Man spricht deshalb von einer 8-Bit-Farbtiefe.

Farbbilder

Um Farbbilder zu verstehen, müssen wir uns zuerst damit auseinandersetzen, wie wir Menschen Farbe überhaupt wahrnehmen. Auf unserer Netzhaut befinden sich drei Arten von Zapfen, die auf unterschiedliche Bereiche des sichtbaren Lichts besonders empfindlich reagieren. Vereinfacht sprechen wir dabei von Rot, Grün und Blau (**RGB**). Unser Gehirn verarbeitet die Signale dieser drei Zapfentypen und erzeugt daraus unsere Farbwahrnehmung.

Es gibt auch andere Farbräume abgesehen von RGB, von denen Sie vielleicht gehört haben (z.B. im Bildnerischen Gestalten). CMYK beschreibt Cyan, Magenta, Yellow und Key/Black.

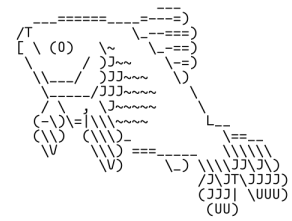


Abbildung 11: Ein Fisch in ASCII-Art.



Abbildung 12: Fotorealitätsnahe Bilder sind heute auf fast allen Geräten möglich.



Abbildung 13: Digital gezeichnete Illustrationen ebenfalls.

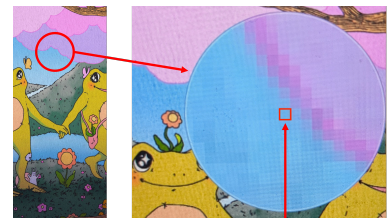


Abbildung 14: Beim Reinzoomen kann man die einzelnen Pixel erkennen.

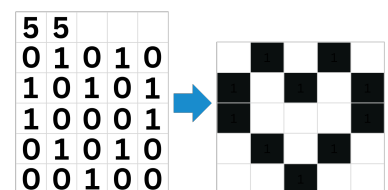


Abbildung 15: Beim .pbm zeigen die oberen zwei Zahlen jeweils Breite und Höhe der Grafik an.

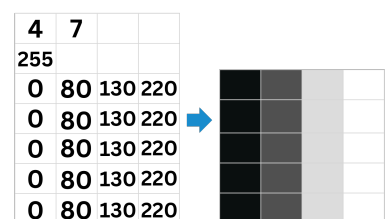


Abbildung 16: Der maximale Wert bei einem Graustufenbild ist 255.

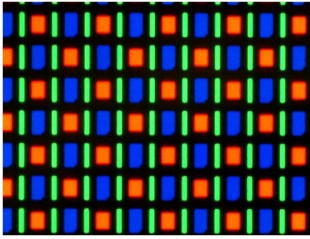


Abbildung 17: Mikroskopische Ansicht eines Bildschirms.

Bei der Darstellung von Farben auf Bildschirmen wird dieses Prinzip unserer empfindlichen Farbkanäle aufgegriffen: Durch die Kombination von rotem, grünem und blauem Licht können sehr viele verschiedene Farben dargestellt werden. Je nachdem, wie stark die drei Farbanteile sind, ergibt sich eine andere Farbe für das bestimmte Pixel.

Bei der Farbtiefe benutzen wir eine ähnliche Abstufung wie bei Graustufenbildern: Pro Farbkanal nimmt man standardgemäss 256 mögliche Werte. Das heisst für einen Pixel hat man $256 \text{ (rot)} \cdot 256 \text{ (grün)} \cdot 256 \text{ (blau)} = 16'777'216$ mögliche verschiedene Farben. Als Farbtiefe bezeichnet man alle möglichen Werte für einen Pixel und weil $2^8 = 256$, bedeutet das also über alle 3 Farbkanäle spricht man von 24 Bit Farbtiefe (8 Bit/Kanal).

24 Bit Farbtiefe wird auch **True Color** genannt. Diese Unterteilung in über 16 Mio Farben ist so fein, dass unser Auge nicht alle Farbtöne auseinander halten kann.

Das heisst aber nicht, dass alle Bilder auf dem Bildschirm unbedingt mit 24 Bit Farbtiefe abgespeichert werden müssen. Möglich ist auch eine gröbere Unterteilung, z.B. 12 Bit Farbtiefe (4Bit/Kanal).

Beispiel True Color

Also mit 8 Bit lässt sich die Farbintensität eines Kanals wie folgt einstellen:

- 0000'0000 : Farbe kommt gar nicht vor
- 1111'1111 : Farbe «ist voll aufgedreht»

Ein paar Farbbeispiele



Links sehen Sie ein Bild bestehend aus 3 Pixeln: ein roter, ein grüner und ein blauer Pixel. Je nachdem, welche Farbtiefe man definiert, muss man das Bild anders codieren. Die Codierung der Pixel wird jeweils mit einem kleinen Abstand dazwischen einfacher sichtbar gemacht.

3 Bit Farbtiefe

Jeder Farbkanal hat jeweils 1 Bit. 0: Farbe kommt nicht vor, 1: sie kommt vor. Es ergibt sich für die Codierung also folgende 3 Tripel:

100 010 001

6 Bit Farbtiefe

Nun hat jeder Farbkanal jeweils 2 Bit für die Codierung. Das heisst, die Werte 00, 01, 10 und 11. Für das Bild erhalten wir:

110000 001100 000011

24 Bit Farbtiefe

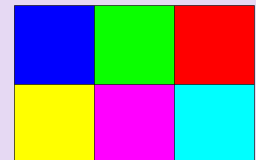
Analog wäre die Codierung nun mit jeweils 8 Bit pro Kanal. Da die Codierung binär etwas lang wäre, schreiben wir es z.B. hexadezimal:

FF0000 00FF00 0000FF

Man kann natürlich auch Farben mischen und muss nicht reines Rot, Grün oder Blau nehmen. Schauen wir uns daher das Bild rechts an und wählen die Farbtiefe von 12 Bit.

Die erste Zeile des Bildes ist analog wie beim vorherigen Bild. Die zweite Zeile besteht aber aus Mischungen der RGB-Farben. Gelb ergibt sich aus Rot und Grün, Magenta aus Rot und Blau, Cyan aus Grün und Blau. Daher ergibt sich für die 12 Bit Codierung des Bildes:

000000001111 000011110000 111100000000
111111110000 111100001111 000011111111



Jetzt werden vielleicht einige verwirrt sein, seit wann denn Rot und Grün zusammen Gelb gibt... Da wir hier von Bildschirmen und Farben reden, handelt es sich um sogenannte **Lichtfarben** (weil der Bildschirm ja Licht aussendet) und nicht die Pigmentfarben, die Sie aus dem bildnerischen Gestalten kennen. Man nennt dies das **additive Farbmodell**.

Also je mehr Licht, desto heller. Wenn alle Farbkanäle voll offen sind (alle Farben zu 100% präsent) ergibt sich weiss. Wenn alle Farben abgedreht sind, ergibt sich schwarz (Absenz von Licht).

Platzbedarf eines Bildes

Unter **Platzbedarf** kann man auch die Bildqualität verstehen (falls es sich um ein unkomprimiertes Bild handelt). Generell kann man sagen, dass je kleiner die Farbtiefe des Bildes, umso schlechter die Qualität und umso weniger Speicher wird benötigt. Das ist auch intuitiv: Weniger Farbtiefe bedeutet, es braucht weniger Bit, um eine Farbe zu definieren.

Um Speicherplatz zu reduzieren, kann man also die Farbtiefe reduzieren. Auch gibt es kleine Tricks, wie z.B. bei der Farbtiefe 8 Bit (was ja eigentlich nicht gleichmässig auf die 3 Kanäle aufzuteilen ist), benutzt man jeweils 3 Bit für Rot

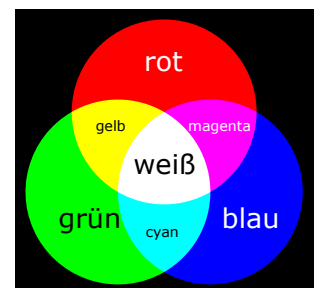


Abbildung 18: Das additive Farbmodell mit den Lichtfarben.

und Grün aber nur 2 Bit für Blau. Der blaue Kanal hier wird benachteiligt ohne grosse Konsequenzen: Unser Auge ist etwas weniger empfindlich bei Blauwerten und drum kann man die schlechtere Qualität gut in Kauf nehmen.

Standardgemäss enthalten Rastergrafikformate jeweils:

- einen kurzen Anfangsteil (Header), der Informationen enthält wie die Dimension des Bildes und Farbtiefe
- restlicher Teil bestehend aus Bits.

Header: Breite, Höhe, Farbtiefe Rest: Bits, die das Bild definieren

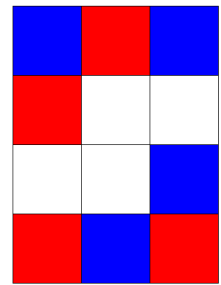
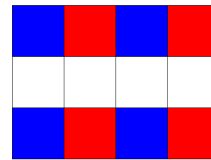
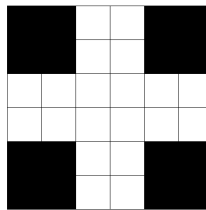
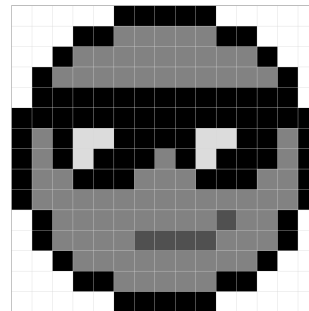
Der Platzbedarf eines Bildes berechnet sich aus seiner Auflösung und der verwendeten Farbtiefe (und einigen Bytes für den Header). Wenn wir ein Bild betrachten mit einer Auflösung von 400x600 Pixel und eine Farbtiefe von 24 Bit anschauen, berechnet sich der Platzbedarf wie folgt: $400 \times 600 \times 24 = 0.72 \text{ MB}$

Gleiche Bits ergeben verschiedene Bilder

Betrachten Sie die folgende Bitfolge und überlegen Sie, welches Bild es codieren soll:

00110000110011111111111001100001100

Welches der 3 Bilder rechts entspricht der Bitfolge?

[illegible]

Lösungen

Lösung 1

$\frac{150}{7.5} \text{ ms} = 20 \text{ ms}$ für die ersten 6 Impulse und 30ms für den Stoppimpuls.

Eine Sekunde hat 1000ms, dh. pro Sekunde können $\frac{1000}{150} \approx 6.67$ Zeichen geschrieben werden. Pro Minute bedeutet das 400 Zeichen. Das ist nicht die schnellste Technologie aber sicher auch nicht so langsam, wie man sich das vielleicht vorstellt!

Lösung 2

- Bei 5 Bit gibt es $2^5 = 32$ Möglichkeiten. Das heisst 32 verschiedene Zeichen sind möglich.
- Nein. 32 Zeichen reichen nicht aus, um beispielsweise alle Gross- und Kleinbuchstaben, Zahlen, Satzzeichen und weitere Sonderzeichen (Emojis?) darzustellen.
- G (— ·) und E (·) zusammen ergibt — · ·
- Nein. Die Folge — · · könnte als GE verstanden werden, aber auch als Buchstabe Z. Der Empfänger weiss ohne zusätzliche Information also nicht, wo das G endet und das E beginnt. Es müsste also eine Pause oder ein zusätzliches Trennsignal zwischen den Zeichen gesendet werden.
- Eine geeignete (moderne) Zeichencodierung müsste genügend verschiedene Zeichen darstellen können und eindeutig sein, damit der Empfänger weiss, wo ein Zeichen beginnt und endet.

Lösung 3

77 97 99 104 116 32 73 110 102 111 114 109 97 116
105 107 32 83 112 97 115 115 63

Lösung 4

1001000 1100101 1110101 1110100 1100101 0100000
1101001 1110011 1110100 0100000 1100101 1110011
0100000 1110011 1100011 1101000 1101111 1100101
1101110

Für das ö haben wir kein geeignetes ASCII-Zeichen gesehen, darum haben wir es mit oe ersetzt. Für das Emoji haben wir aber bisher keine Lösung...

Lösung 5

$2^7 = 128$ und $2^8 = 256$. Es kommen also 128 neue Zeichen dazu (doppelt so viele wie vorher).

Lösung 6

Das kyrillische Alphabet, welches z.B. für die ukrainische, bulgarische oder auch russische Sprache verwendet wird, ist nicht in diesen Zeichen enthalten. Ein anderes Beispiel wäre auch das griechische Alphabet.

Lösung 7

48 69 6E 74 65 72 20 31 30 30 20 48 65 63 6B 65 6E
20 68 6F 63 6B 65 6E 20 31 30 30 20 48 61 73 65 6E

Lösung 8

Mit „nicht druckbaren Steuerzeichen“ sind Zeichen in der ASCII-Tabelle gemeint, die nicht als sichtbares Zeichen auf dem Bildschirm dargestellt oder ausgedruckt werden. Sie dienen stattdessen dazu, bestimmte Funktionen zu steuern, zum Beispiel einen Zeilenumbruch oder das Ende einer Eingabe zu kennzeichnen.

Lösung 9

- dog
- :{}
- $28 + 37 = 65$

Lösung 10

- 0100'1000
- 0110'0101
- 0111'1001
- 1111'0000 1001'1111 1001'1000 1000'1001
- 1100'0011 1010'0100
- 0110'1000
- 0110'1101
- 1111'0000 1001'1111 1000'1111 1000'0001

Decodiert würde der Satz bedeuten: Hey 😊 ähm 🌀

Lösung 11

- 0101'0001
- 1111'0000 1001'1111 1001'0010 1010'1001
- 1110'0010 1001'1001 1010'0101
- 1100'0011 1011'1100

Lösung 12

- 10'0000 1010'1100
- 1 1111'0100 0011'0110

Lösung 13

- 0'0001 1111'0011 1100'0001 → 01F3CI → 🚩
- 000 1110'0100 → 0E4 → ä

Lösung 14

- Gemüse
- voilà